

Why Disable a Test?

- **Disabling tests should be avoided** unless absolutely necessary
- A **failing test** usually **indicates a real problem** that **should be fixed**
- **Disabled tests** can **hide actual issues** in the codebase
- Only **disable temporarily** when there is a **clear** and **justified reason**

 **by Rodrigo Martins Pagliares**

Last update: 11/26/2025



Method 1: Removing @Test Annotation

- The **simplest way** to **disable** a **test** is to **comment out** the `@Test` annotation
- **Without** this **annotation**, the **method** is **no longer treated** as a **test**
- The **method** does **not** run and does **not** appear in the **test report**
- Useful for **quick temporary disabling** during **experimentation** or **debugging**



Method 1: Removing @Test Annotation (contd.)

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The file "CalculatorTest.java" is open, showing the following code:

```
9 public class CalculatorTest {
67
68     @DisplayName("Trying to divide by zero") no usages
69     // @Test
70     public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {
71         System.out.println("Running division by zero");
72         fail("Not yet implemented");
73     }
74 }
75
```

Below the code editor, the "Run" tab is active, showing the test results for "CalculatorTest". The test suite "Test operations in Calculator class (org.example)" passed with a total time of 14 ms. The individual tests are:

- Test 10/2 = 5 (1 ms)
- Test 113-13=100 (13 ms)

The test output shows the following sequence of events:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...
Executing @BeforeAll method
Executing @BeforeEach method
Test 113-13=100
Executing @AfterEach method
Executing @BeforeEach method
Test 10/2 = 5
Executing @AfterEach method
Executing @AfterAll method
```

Method 2: Using @Disabled Annotation

- **JUnit** offers a **dedicated @Disabled annotation** to **deactivate tests**
- The test is **skipped** but still **visible** in **test reports**
- You can add a reason (e.g., "TODO: still need to work on this")
- **Recommended approach** because it preserves test visibility and reminders



Method 2: Using @Disabled Annotation (contd.)

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The file "CalculatorTest.java" is open, showing a Java class with two test methods. The first method, `testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturn0`, is partially visible. The second method, `testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException`, is annotated with `@Disabled("TODO: Further work is required here.")` and `@DisplayName("Trying to divide by zero")`. The test body prints "Running division by zero" and then fails with the message "Not yet implemented".

Below the code editor, the "Run" tab shows the execution results for "CalculatorTest". It indicates that 2 tests passed, 1 was ignored, and 3 tests were run in total, taking 12 ms. The test results list shows:

- ✓ Test 10/2 = 5 (12 ms)
- ✓ Test 113-13=100 (12 ms)
- Trying to divide by zero (ignored)

The output window displays the message: "TODO: Further work is required here.".

```
9      public class CalculatorTest {
56          public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturn0() {
66      }
67
68      @Disabled("TODO: Further work is required here.")
69      @DisplayName("Trying to divide by zero")
70      @Test
71      public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {
72          System.out.println("Running division by zero");
73          fail("Not yet implemented");
74      }
75  }
```

Run CalculatorTest x

✓ 2 tests passed, 1 ignored 3 tests total, 12 ms

- ✓ Test 10/2 = 5 12 ms
- ✓ Test 113-13=100 12 ms
- Trying to divide by zero

TODO: Further work is required here.

CalculatorMavenProject > src > test > java > org > example > CalculatorTest 68:52 LF UTF-8 4 spaces

Comparing Both Approaches

Removing @Test

hidden test, no visibility, easy to forget

Using @Disabled

test stays **listed** and **clearly marked** as **disabled**

Teams benefit from transparency when tests are visible in reports

Best practice: prefer @Disabled over removing the annotation



Method 3 (Alternative): Make the test fail miserably instead of disabling it

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The editor displays the file "CalculatorTest.java" with the following code:

```
9      public class CalculatorTest {
56          public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturn0
66      }
67
68      @DisplayName("Trying to divide by zero")
69      @Test
70      public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {
71          System.out.println("Running division by zero");
72          fail("Not yet implemented");
73      }
74  }
```

The "Run" tab at the bottom shows the execution results for "CalculatorTest". It indicates that 1 test failed and 2 tests passed, totaling 3 tests in 17 ms. The failed test is "Trying to divide by zero" (4 ms). The output of the failed test is as follows:

```
Executing @BeforeEach method
Running division by zero
Executing @AfterEach method

org.opentest4j.AssertionFailedError: Not yet implemented
> <2 internal lines>
    at org.example.CalculatorTest.testPerformIntegerDivision_WhenDividendIsDi
```

The status bar at the bottom shows the file path: "CalculatorMavenProject > src > test > java > org > example > CalculatorTest". The encoding is UTF-8, and the line length is 72:34.

```
fail("Not yet implemented");
```

Conclusion

- ☐ Disabling tests should be rare and carefully justified
- ☐ Prefer `@Disabled` to keep visibility and reminders in reports
- ☐ Never disable tests simply to "make the build pass"
- ☐ A failing test is a valuable signal that should be investigated





Open Floor Discussion

When would disabling a test be acceptable in your project?

Does your team have rules about temporarily disabling tests?

Should disabled tests require comments or issue tracking references?